

GSOI 2026 暑期常规赛

第五试题解

Ehundategh & tfbz

Gioush 大队

2026 年 7 月

目录

T1 我流奥义 (smoke)

T2 星语楠尘 (stardust)

T3 日月风随 (solwind)

T4 推翻暴政 (revolt)

吐槽时间

Contents

T1 我流奥义 (smoke)

T2 星语楠尘 (stardust)

T3 日月风随 (solwind)

T4 推翻暴政 (revolt)

吐槽时间

数据点 1 ~ 5 25Pts

- ▶ 这个时候单组内 $\sum n \leq 2000$ 。先删去圆外的点；若原点出现，答案直接为 1。否则枚举每个保留下来的点，再与先前出现的方向逐一比较，判断两组坐标是否同向。

数据点 1 ~ 5 25Pts

- ▶ 这个时候单组内 $\sum n \leq 2000$ 。先删去圆外的点；若原点出现，答案直接为 1。否则枚举每个保留下来的点，再与先前出现的方向逐一比较，判断两组坐标是否同向。
- ▶ 两个非零向量同向，当且仅当叉积为 0 且点积为正。这样直接去重的时间复杂度为 $\mathcal{O}(n^2)$ ，并且已经说明真正需要保存的不是距离，而是从原点出发的有向射线。

数据点 6 ~ 10 25Pts

- 这一档满足 $\max(|x_i|, |y_i|) \leq 2000$ 。考虑将 (x_i, y_i) 同时除以 $\gcd(|x_i|, |y_i|)$ ，所得最简整数对唯一表示一条有向射线，并且负号仍然保留，因此相反方向不会被错误合并。

数据点 6 ~ 10 25Pts

- ▶ 这一档满足 $\max(|x_i|, |y_i|) \leq 2000$ 。考虑将 (x_i, y_i) 同时除以 $\gcd(|x_i|, |y_i|)$ ，所得最简整数对唯一表示一条有向射线，并且负号仍然保留，因此相反方向不会被错误合并。
- ▶ 使用带时间戳的二维数组记录最简整数对，读入时便能够完成去重。每个点只处理一次，时间复杂度为 $\mathcal{O}(n)$ ；这一档给出了正解中最关键的方向标准化。

数据点 11 ~ 15 25Pts

- ▶ 特殊性质保证所有点都在烟雾半径以内，因此只剩下原点与同向遮挡两种情况。仍然将每个非零坐标约分成最简整数对，再排序并统计不同整数对的数量。

数据点 11 ~ 15 25Pts

- ▶ 特殊性质保证所有点都在烟雾半径以内，因此只剩下原点与同向遮挡两种情况。仍然将每个非零坐标约分成最简整数对，再排序并统计不同整数对的数量。
- ▶ 这一档已经完成了正解的主体，时间复杂度为 $\mathcal{O}(n \log n)$ ；最后一档只需要恢复圆的边界判断。距离平方可能达到 10^{18} ，乘法必须先转成 long long。

数据点 16 ~ 20 正解

- ▶ 对每个点先比较 $x_i^2 + y_i^2$ 与 r^2 ，圆外的点不会被烟雾触及，直接删除。若存在原点，题意已经规定只有原点会被标记，当前测试立即输出 1。

数据点 16 ~ 20 正解

- ▶ 对每个点先比较 $x_i^2 + y_i^2$ 与 r^2 ，圆外的点不会被烟雾触及，直接删除。若存在原点，题意已经规定只有原点会被标记，当前测试立即输出 1。
- ▶ 其余点全部除以横纵坐标绝对值的最大公约数，排序后统计不同最简整数对即可。总时间复杂度为 $\mathcal{O}(n \log n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

正解 100Pts: 方向标准化

```
1 struct point{
2     int x,y;
3 }Point[1000010];
4
5 bool cmp(point a,point b){
6     if(a.x==b.x) return a.y<b.y;
7     return a.x<b.x;
8 }
9
10 void Solve(){
11     int Count=0;bool Tag=1;
12     scanf("%d%d",&n,&r);
13     for(int i=1;i<=n;i++){
14         scanf("%d%d",&In1,&In2);
15         if(1ll*In1*In1+1ll*In2*In2>1ll*r*r){
16             i--,n--;
17             continue;
18         }
```

正解 100Pts: 统计答案

```
1         if(In1==0&&In2==0){Tag=0;i--,n--;continue;}
2         int Gcd=__gcd(abs(In1),abs(In2));
3         In1/=Gcd;In2/=Gcd;
4         Point[i]={In1,In2};
5     }
6     if (!Tag) {puts("1");return;}
7     sort(Point+1,Point+n+1,cmp);
8     for(int i=1;i<=n;i++){
9         if((Point[i].x!=Point[i-1].x||Point[i].y!=Point[i-1].y)||(i==1))
10            Count++;
11     }
12     printf("%d\n",Count);
13     return;
14 }
```

Contents

T1 我流奥义 (smoke)

T2 星语楠尘 (stardust)

T3 日月风随 (solwind)

T4 推翻暴政 (revolt)

吐槽时间

数据点 1 ~ 5 25Pts

- 枚举每一座星塔作为根，再从根出发遍历整棵树，检查每个非根结点对应的子树是否同色。单次检查为 $\mathcal{O}(n)$ ，枚举所有根的总时间复杂度为 $\mathcal{O}(n^2)$ 。

数据点 1 ~ 5 25Pts

- ▶ 枚举每一座星塔作为根，再从根出发遍历整棵树，检查每个非根结点对应的子树是否同色。单次检查为 $\mathcal{O}(n)$ ，枚举所有根的总时间复杂度为 $\mathcal{O}(n^2)$ 。
- ▶ 更直接地看，一条两端颜色不同的边无论朝哪个方向定根，若它不与根相连，就一定落在某个非根子树内部，使这个子树出现两种颜色。因此合法根必须覆盖全部异色边。

数据点 6 ~ 10 25Pts

- ▶ 特殊性质 A 保证树是一条链。链上的异色边必须全部拥有同一个公共端点：若没有异色边，任意点都合法；若所有异色边都与某个点相连，这个点便是合法根。

数据点 6 ~ 10 25Pts

- ▶ 特殊性质 A 保证树是一条链。链上的异色边必须全部拥有同一个公共端点：若没有异色边，任意点都合法；若所有异色边都与某个点相连，这个点便是合法根。
- ▶ 按链的顺序扫描颜色变化处即可，时间复杂度为 $\mathcal{O}(n)$ 。这一档将上一档的必要条件推进成了充要条件，也正是完整树上需要使用的判定。

数据点 11 ~ 15 25Pts

- ▶ 特殊性质 B 保证只要有解，编号 1 就是一个合法根。因此只需检查每一条异色边是否都与 1 相连；全部满足就输出 1，否则输出 NO。

数据点 11 ~ 15 25Pts

- ▶ 特殊性质 B 保证只要有解，编号 1 就是一个合法根。因此只需检查每一条异色边是否都与 1 相连；全部满足就输出 1，否则输出 NO。
- ▶ 这个检查与树的形态无关，只需要遍历边，时间复杂度为 $\mathcal{O}(n)$ 。它同时给出了正解中验证一个候选根的方法。

数据点 16 ~ 20 正解

- ▶ 找到第一条异色边 (u, v) 。由于任意合法根都必须与这条边相连，所以候选只可能是 u 或 v ；分别检查所有异色边是否都与候选相连即可。

数据点 16 ~ 20 正解

- ▶ 找到第一条异色边 (u, v) 。由于任意合法根都必须与这条边相连，所以候选只可能是 u 或 v ；分别检查所有异色边是否都与候选相连即可。
- ▶ 若不存在异色边，整棵树同色，任意根都合法。反之，某个候选覆盖了全部异色边时，删去根以后每个连通块内部都没有异色边，所以每个非根子树同色，充分性也成立。总时间复杂度为 $\mathcal{O}(n)$ 。

正解 100Pts: 检查候选

```
1 bool Check(int Root) {
2     for (int i=1;i<=n-1;i++)
3         if (Col[Edge[i].St]!=Col[Edge[i].Ed]&&Edge[i].St!=Root&&Edge[i].
4             Ed!=Root) return 0;
5     return 1;
6 }
7 void Solve() {
8     scanf("%d",&n);
9     for (int i=1;i<=n-1;i++) {
10         scanf("%d%d",&Edge[i].St,&Edge[i].Ed);
11     }
12     for (int i=1;i<=n;i++) scanf("%d",&Col[i]);
```

正解 100Pts: 两个端点

```
1  int u=0,v=0;
2  for (int i=1;i<=n-1;i++) {
3      if (Col[Edge[i].St]!=Col[Edge[i].Ed]) {
4          u=Edge[i].St;v=Edge[i].Ed;
5          break;
6      }
7  }
8  if (!u) puts("YES"),puts("1");
9  else if (Check(u)) puts("YES"),printf("%d\n",u);
10 else if (Check(v)) puts("YES"),printf("%d\n",v);
11 else puts("NO");
12 return;
13 }
```

- Cena 的 SPJ 接受任意合法根，因此不要求与标准输出中的编号相同。

Contents

T1 我流奥义 (smoke)

T2 星语楠尘 (stardust)

T3 日月风随 (solwind)

T4 推翻暴政 (revolt)

吐槽时间

数据点 1 ~ 6 24Pts

- 枚举举行仪式的位置 p 。固定 p 以后，峰顶没有必要提前削弱，保留高度取 $b_p = h_p$ ；向左依次令 $b_i = \max(0, \min(h_i, b_{i+1} - 1))$ ，向右对称处理。

数据点 1 ~ 6 24Pts

- ▶ 枚举举行仪式的位置 p 。固定 p 以后，峰顶没有必要提前削弱，保留高度取 $b_p = h_p$ ；向左依次令 $b_i = \max(0, \min(h_i, b_{i+1} - 1))$ ，向右对称处理。
- ▶ 仪式前削弱的消耗为 $\sum (h_i - b_i)$ ，仪式本身还要投入 h_p ，因此 $C_p = \sum (h_i - b_i) + h_p$ 。每个峰顶线性模拟，总时间复杂度为 $\mathcal{O}(n^2)$ 。

数据点 7 ~ 12 24Pts

- 特殊性质保证所有 $h_i = H$ 。固定峰顶以后，向一侧最多保留 $H, H-1, \dots, 1$ ；记

$$\begin{aligned} F(H, L) &= \sum_{j=0}^{\min(H, L)-1} (H-j) \\ &= \frac{(2H - \min(H, L) + 1) \min(H, L)}{2}. \end{aligned}$$

数据点 7 ~ 12 24Pts

- 特殊性质保证所有 $h_i = H$ 。固定峰顶以后，向一侧最多保留 $H, H-1, \dots, 1$ ；记

$$\begin{aligned} F(H, L) &= \sum_{j=0}^{\min(H, L)-1} (H-j) \\ &= \frac{(2H - \min(H, L) + 1) \min(H, L)}{2}. \end{aligned}$$

- 峰顶为 p 时，保留总和为 $F(H, p) + F(H, n-p+1) - H$ ，代入上一档的代价式即可。枚举峰顶的时间复杂度为 $\mathcal{O}(n)$ 。

数据点 13 ~ 19 28Pts: 限高 DP 状态

- 这一档满足 $h_i \leq 100$ 。设 $d_{i,v}$ 表示只考虑前 i 个位置、保留序列的正数部分严格递增，并且第 i 个位置保留高度恰为 v 时，能够保留的最大总高度；设 $p_{i,v} = \max_{0 \leq u \leq v} d_{i,u}$ 。

数据点 13 ~ 19 28Pts: 限高 DP 状态

- ▶ 这一档满足 $h_i \leq 100$ 。设 $d_{i,v}$ 表示只考虑前 i 个位置、保留序列的正数部分严格递增，并且第 i 个位置保留高度恰为 v 时，能够保留的最大总高度；设 $p_{i,v} = \max_{0 \leq u \leq v} d_{i,u}$ 。
- ▶ 初始化 $d_{0,0} = 0$, $p_{0,v} = 0$ ；当 $v > h_i$ 时令 $d_{i,v} = -\infty$ ，表示该状态不合法。 $v = 0$ 表示这一侧尚未开始保留正数，因而 $d_{i,0} = 0$ 。

数据点 13 ~ 19 28Pts: 完整 DP 方程

► 从左向右的转移与前缀最大值为

$$\begin{cases} d_{i,0} = 0, \\ d_{i,v} = v + p_{i-1,v-1}, & 1 \leq v \leq h_i, \\ d_{i,v} = -\infty, & v > h_i, \end{cases}$$

并在同一行位置上计算

$$p_{i,v} = \max\{p_{i,v-1}, d_{i,v}\}.$$

从右向左完全相同，记所得状态为 $e_{i,v}$ 与后缀最大值 $q_{i,v}$ 。

数据点 13 ~ 19 28Pts: 答案与复杂度

- 峰顶固定为 i 时取 $v = h_i$, 左右两边都计算了一次峰顶, 且仪式另需投入 h_i 。记原高度总和为 S , 则

$$S = \sum_{j=1}^n h_j,$$

$$\text{Ans} = \min_i \{S - d_{i,h_i} - e_{i,h_i} + 2h_i\}.$$

数据点 13 ~ 19 28Pts: 答案与复杂度

- 峰顶固定为 i 时取 $v = h_i$ ，左右两边都计算了一次峰顶，且仪式另需投入 h_i 。记原高度总和为 S ，则

$$S = \sum_{j=1}^n h_j,$$

$$\text{Ans} = \min_i \{S - d_{i,h_i} - e_{i,h_i} + 2h_i\}.$$

- 状态按位置滚动，时间复杂度为 $\mathcal{O}(nH)$ ，空间复杂度为 $\mathcal{O}(H + n)$ 。这正是下一档要消去的高度维度。

数据点 20 ~ 25：去掉高度上界

- ▶ 限高 DP 的瓶颈是枚举峰顶高度。回到第一档，对固定右端点 i ，每个位置能够保留的高度是若干个上界 $h_k - (i - k)$ 的最小值；当新的 h_i 加入以后，一段连续的旧上界会被它整体取代。

数据点 20 ~ 25: 去掉高度上界

- ▶ 限高 DP 的瓶颈是枚举峰顶高度。回到第一档，对固定右端点 i ，每个位置能够保留的高度是若干个上界 $h_k - (i - k)$ 的最小值；当新的 h_i 加入以后，一段连续的旧上界会被它整体取代。
- ▶ 因此维护这些上界发生变化的位置。栈内下标对应的有效高度严格保持，若新位置的斜线已经压过栈顶，就删除栈顶；每个位置至多入栈、出栈各一次。

正解：单调栈维护贡献

- 若栈中相邻断点为 q 与 i ，由 i 控制的这一段长度为 $L = i - q$ ，其保留高度和正是

$$F(h_i, L) = \frac{(2h_i - \min(h_i, L) + 1) \min(h_i, L)}{2}.$$

弹出断点时减去它原来的贡献，压入新断点时加上新的贡献，便得到以 i 为峰顶时左侧能够保留的总和。

正解：单调栈维护贡献

- ▶ 若栈中相邻断点为 q 与 i ，由 i 控制的这一段长度为 $L = i - q$ ，其保留高度和正是

$$F(h_i, L) = \frac{(2h_i - \min(h_i, L) + 1) \min(h_i, L)}{2}.$$

弹出断点时减去它原来的贡献，压入新断点时加上新的贡献，便得到以 i 为峰顶时左侧能够保留的总和。

- ▶ 从左向右与从右向左各做一次，分别得到 $\text{Need}[0][i]$ 与 $\text{Need}[1][i]$ 。最终取两侧削弱代价之和再加峰顶投入，时间复杂度为 $\mathcal{O}(n)$ ，所有和与答案均需使用 long long。

正解 100Pts: 区间贡献

```
1 class Container{
2 public:
3     int Head=1,Tail=0,Pos[MAXN];;
4     long long Sum=0;
5     void Init(){Head=1,Tail=0;}
6     void Push_Back(int a){Pos[++Tail]=a;}
7     void Pop_Back(){Tail--;}
8     int Back(){return Pos[Tail];}
9     bool Empty(){return Head>=Tail;}
10 }Q;
11 long long Calc(long long Height,long long Length){//计算元素贡献
12     Length=min(Length,Height);
13     return 1ll*(Height+Height-Length+1)*Length/2;
14 }
```

正解 100Pts: 从左向右

```
1      for(int i=1;i<=n;i++){
2          PreSum+=Line[i];
3          while(i-Q.Back()>Line[i]-Line[Q.Back()]&&!Q.Empty()){
4              Q.Sum-=Calc(Line[Q.Back()],(long long)(Q.Back()-Q.Pos[Q.
Tail-1]));
5              Q.Pop_Back();
6          }
7          Q.Push_Back(i);
8          Q.Sum+=Calc(Line[Q.Back()],(long long)(Q.Back()-Q.Pos[Q.Tail
-1]));
9          Need[0][i]=PreSum-Q.Sum;
10     }
```

正解 100Pts: 反向与合并

```
1      for(int i=n;i>=1;i--){
2          SuffSum+=Line[i];
3          while(Q.Back()-i>Line[i]-Line[Q.Back()]&&!Q.Empty()){
4              Q.Sum-=Calc(Line[Q.Back()],(long long)(Q.Pos[Q.Tail-1]-Q
5              .Back()));
6              Q.Pop_Back();
7          }
8          Q.Push_Back(i);
9          Q.Sum+=Calc(Line[Q.Back()],(long long)(Q.Pos[Q.Tail-1]-Q.
10         Back()));
11         Need[1][i]=SuffSum-Q.Sum;
12     }
13     for(int i=1;i<=n;i++){
14         if(Need[0][i]+Need[1][i]+Line[i]<Ans) Ans=Need[0][i]+Need
15         [1][i]+Line[i];
16     }
```

Contents

T1 我流奥义 (smoke)

T2 星语楠尘 (stardust)

T3 日月风随 (solwind)

T4 推翻暴政 (revolt)

吐槽时间

数据点 1 ~ 4 16Pts

- ▶ 每一轮渡口恰好有三个人，选择两个人离开也就等价于选择一个人留下。因此直接枚举“留下原来的人、留下第一名新人、留下第二名新人”三种情况，递归处理下一批即可。

数据点 1 ~ 4 16Pts

- ▶ 每一轮渡口恰好有三个人，选择两个人离开也就等价于选择一个人留下。因此直接枚举“留下原来的人、留下第一名新人、留下第二名新人”三种情况，递归处理下一批即可。
- ▶ 一共进行 $(n-1)/2$ 轮，每轮有三种选择；所有人到达后，再把最后留下者独自离开的时间加入答案。时间复杂度为 $\mathcal{O}(3^{(n-1)/2})$ ，而递归参数中真正会影响未来的只有留下者的编号。

数据点 5 ~ 9 20Pts: 状态与初始化

- 递归已经告诉我们状态中只需保留当前留下的人。设 $Dp(u)$ 表示已经处理完当前到达的所有人、留下第 u 名调查员时，此前所有共同离开过程的最少总时间。

数据点 5 ~ 9 20Pts: 状态与初始化

- ▶ 递归已经告诉我们状态中只需保留当前留下的人。设 $Dp(u)$ 表示已经处理完当前到达的所有人、留下第 u 名调查员时，此前所有共同离开过程的最少总时间。
- ▶ 最初渡口只有第 1 名调查员，因此 $Dp(1) = 0$ ，其余状态均为 $+\infty$ 。注意这里还没有计算最后一人独自离开的时间，它只会在所有轮次结束后加入答案。

数据点 5 ~ 9 20Pts: 完整 DP 方程

- 处理新到达的第 $i-1, i$ 名调查员时, 从旧状态 u 出发恰好有三种转移:

$$\begin{cases} \text{Dp}'(u) \leftarrow \min\{\text{Dp}'(u), \text{Dp}(u) + \max(a_{i-1}, a_i)\}, \\ \text{Dp}'(i-1) \leftarrow \min_u\{\text{Dp}(u) + \max(a_u, a_i)\}, \\ \text{Dp}'(i) \leftarrow \min_u\{\text{Dp}(u) + \max(a_u, a_{i-1})\}. \end{cases}$$

其中 Dp' 表示这一轮结束后的新数组, 开始转移前全部初始化为 $+\infty$ 。

数据点 5 ~ 9 20Pts: 答案与复杂度

- 第一种转移留下旧人，另外两种转移分别留下两名新人，三者恰好覆盖全部合法选择。所有人到达后，最后留下的第 u 名调查员还要独自离开，因此

$$\text{Ans} = \min_u \{ \text{Dp}(u) + a_u \}.$$

数据点 5 ~ 9 20Pts: 答案与复杂度

- ▶ 第一种转移留下旧人，另外两种转移分别留下两名新人，三者恰好覆盖全部合法选择。所有人到达后，最后留下的第 u 名调查员还要独自离开，因此

$$\text{Ans} = \min_u \{ \text{Dp}(u) + a_u \}.$$

- ▶ 每一轮枚举所有旧状态即可完成三种转移，时间复杂度为 $\mathcal{O}(n^2)$ ，使用新旧两个数组时空复杂度为 $\mathcal{O}(n)$ 。

数据点 10 ~ 14 20Pts

- 特殊性质 A 保证 $a_1 \leq a_2 \leq \dots \leq a_n$ 。设旧人的时间为 z ，新到达的两人为 $x \leq y$ ，并且 $z \leq x$ 。若留下旧人，本轮支付 y ；若改为留下 x 或 y ，虽然本轮可能少支付一些，却把最终留下者至少从 z 提高到了 x 。

数据点 10 ~ 14 20Pts

- ▶ 特殊性质 A 保证 $a_1 \leq a_2 \leq \dots \leq a_n$ 。设旧人的时间为 z ，新到达的两人为 $x \leq y$ ，并且 $z \leq x$ 。若留下旧人，本轮支付 y ；若改为留下 x 或 y ，虽然本轮可能少支付一些，却把最终留下者至少从 z 提高到了 x 。
- ▶ 将第一次更换留下者的选择与最终独自渡河合并比较，更换产生的节省不超过最终代价的增加，因此始终留下第 1 人最优。答案为 $a_1 + a_3 + a_5 + \dots + a_n$ ，时间复杂度为 $\mathcal{O}(n)$ 。

数据点 15 ~ 19 20Pts

- ▶ 特殊性质 B 保证序列中至多有 100 种不同时间。将相同 a_i 合并为同一个状态，直接按照完整 DP 方程枚举旧时间 z ，每轮至多检查 100 个状态。

数据点 15 ~ 19 20Pts

- ▶ 特殊性质 B 保证序列中至多有 100 种不同时间。将相同 a_i 合并为同一个状态，直接按照完整 DP 方程枚举旧时间 z ，每轮至多检查 100 个状态。
- ▶ 时间复杂度为 $\mathcal{O}(nV)$ ，其中 $V \leq 100$ 。这一档保留了正解的状态，只把“对全部旧状态取最小值”限制在较小值域内。

数据点 20 ~ 25: 拆开转移

- 回到完整 DP，后两种转移都在反复计算同一种询问。对于给定的渡河时间 v ，定义

$$\text{Calc}(v) = \min_u \{ \text{Dp}(u) + \max(a_u, v) \}.$$

数据点 20 ~ 25: 拆开转移

- 回到完整 DP，后两种转移都在反复计算同一种询问。对于给定的渡河时间 v ，定义

$$\text{Calc}(v) = \min_u \{ \text{Dp}(u) + \max(a_u, v) \}.$$

- 根据 $a_u \leq v$ 与 $a_u > v$ 拆开最大值，便有

$$\text{Calc}(v) = \min \begin{cases} v + \min_{a_u \leq v} \text{Dp}(u), \\ \min_{a_u > v} (\text{Dp}(u) + a_u). \end{cases}$$

于是原来枚举全部状态的瓶颈，变成了一个前缀最小值和一个后缀最小值。

数据点 20 ~ 25: 减去全局增量

- 第一种转移会给所有旧状态统一加上 $\max(a_{i-1}, a_i)$ 。令 Tag 记录这些公共增量，并在线段树中保存

$$\text{Val}(u) = \text{Dp}(u) - \text{Tag}.$$

这样留下旧人时只需增加 Tag，不再需要修改每一个旧状态。

数据点 20 ~ 25: 减去全局增量

- 第一种转移会给所有旧状态统一加上 $\max(a_{i-1}, a_i)$ 。令 Tag 记录这些公共增量，并在线段树中保存

$$\text{Val}(u) = \text{Dp}(u) - \text{Tag}.$$

这样留下旧人时只需增加 Tag，不再需要修改每一个旧状态。

- 将 $\text{Dp}(u) = \text{Val}(u) + \text{Tag}$ 代入前面的拆式，得到

$$\text{Calc}(v) = \text{Tag} + \min \begin{cases} v + \min_{a_u \leq v} \text{Val}(u), \\ \min_{a_u > v} (\text{Val}(u) + a_u). \end{cases}$$

数据点 20 ~ 25: 离散化与维护信息

- ▶ 所有可能留下的人都来自原序列，因此只需把全部 a_i 复制到 Mapping 中。Discrete() 排序并去重，Find() 返回一个渡河时间在线段树中的位置。

数据点 20 ~ 25: 离散化与维护信息

- ▶ 所有可能留下的人都来自原序列，因此只需把全部 a_i 复制到 Mapping 中。Discrete() 排序并去重，Find() 返回一个渡河时间在线段树中的位置。
- ▶ 对于每个值域区间，令 Min 表示区间内 $\text{Val}(u)$ 的最小值，令 MinSum 表示区间内 $\text{Val}(u) + a_u$ 的最小值。于是 Min 的前缀询问与 MinSum 的后缀询问正好组成 $\text{Calc}(v)$ 。

数据点 20 ~ 25: 叶子与相同时间

- ▶ 若要插入一个渡河时间为 a_u 、相对代价为 x 的状态，就在对应叶子处分别用 x 更新 Min、用 $x + a_u$ 更新 MinSum，父结点只需对左右儿子取最小值。

数据点 20 ~ 25: 叶子与相同时间

- ▶ 若要插入一个渡河时间为 a_u 、相对代价为 x 的状态，就在对应叶子处分别用 x 更新 Min、用 $x + a_u$ 更新 MinSum，父结点只需对左右儿子取最小值。
- ▶ 若多名调查员的渡河时间相同，他们参与未来转移和最终独自离开的代价完全一致。因此同一叶子只保留最小的 Val 即可，这也解释了代码中两次更新都使用取最小值而不是覆盖。

数据点 20 ~ 25: 每轮顺序

- 处理第 $i-1, i$ 名调查员时, 必须先查旧状态集合上查询

$$\begin{aligned}\text{Left} &= \text{Calc}(a_i), \\ \text{Right} &= \text{Calc}(a_{i-1}).\end{aligned}$$

它们分别是留下第 $i-1$ 人和留下第 i 人的真实最小代价。

数据点 20 ~ 25: 每轮顺序

- 处理第 $i-1, i$ 名调查员时, 必须先在此旧状态集合上查询

$$\begin{aligned}\text{Left} &= \text{Calc}(a_i), \\ \text{Right} &= \text{Calc}(a_{i-1}).\end{aligned}$$

它们分别是留下第 $i-1$ 人和留下第 i 人的真实最小代价。

- 查询完成后, 再令 $\text{Tag} += \max(a_{i-1}, a_i)$, 完成所有旧状态的第一种转移; 最后插入 Left-Tag 与 Right-Tag 。若提前增加 Tag 或提前插入新人, 就会让这一轮的新状态错误地再次参与本轮转移。

数据点 20 ~ 25: 答案

- 全部轮次结束后, 留下第 u 人的完整代价为

$$\text{Dp}(u) + a_u = \text{Tag} + \text{Val}(u) + a_u.$$

根结点的 MinSum 已经维护了所有 $\text{Val}(u) + a_u$ 的最小值, 因此

$$\text{Ans} = \text{Tag} + \text{Node}[1].\text{MinSum}.$$

数据点 20 ~ 25: 答案

- 全部轮次结束后，留下第 u 人的完整代价为

$$\text{Dp}(u) + a_u = \text{Tag} + \text{Val}(u) + a_u.$$

根结点的 MinSum 已经维护了所有 $\text{Val}(u) + a_u$ 的最小值，因此

$$\text{Ans} = \text{Tag} + \text{Node}[1].\text{MinSum}.$$

- 每轮进行两次区间询问和两次单点更新，总时间复杂度为 $\mathcal{O}(n \log n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。状态值、答案、Tag 与无穷大都必须使用 long long。

样例推演：第一轮

- 考虑 $a = (4, 2, 7, 3, 6)$ 。初始只有状态 $Dp(1) = 0$ ；处理第 2, 3 人以后，留下三个人分别对应

u	1	2	3
$Dp(u)$	7	7	4

例如留下第 3 人时，第 1, 2 人共同离开，代价为 $\max(4, 2) = 4$ 。

样例推演：第一轮

- 考虑 $a = (4, 2, 7, 3, 6)$ 。初始只有状态 $Dp(1) = 0$ ；处理第 2, 3 人以后，留下三个人分别对应

u	1	2	3
$Dp(u)$	7	7	4

例如留下第 3 人时，第 1, 2 人共同离开，代价为 $\max(4, 2) = 4$ 。

- 这个表也能直接检查三种转移：留下旧人支付 $\max(2, 7) = 7$ ，留下第 2 人支付 $\max(4, 7) = 7$ ，留下第 3 人支付 $\max(4, 2) = 4$ 。

样例推演：第二轮与答案

- 接着处理第 4, 5 人。旧状态统一增加 $\max(3, 6) = 6$ ；同时 $\text{Calc}(6) = 11$, $\text{Calc}(3) = 10$ ，所以

u	1	2	3	4	5
$\text{Dp}(u)$	13	13	10	11	10

样例推演：第二轮与答案

- ▶ 接着处理第 4, 5 人。旧状态统一增加 $\max(3, 6) = 6$ ；同时 $\text{Calc}(6) = 11$, $\text{Calc}(3) = 10$ ，所以

u	1	2	3	4	5
$\text{Dp}(u)$	13	13	10	11	10

- ▶ 最后分别加上留下者独自离开的时间，得到 17, 15, 17, 14, 16，故最少总时间为 14。其中一种最优过程是先让第 1, 2 人离开，再让第 3, 5 人离开，最后第 4 人独自离开。

正解 100Pts: 离散化与线段树信息

```
1 void Discrete() {sort(Mapping+1,Mapping+n+1);m=unique(Mapping+1,Mapping+
    n+1)-Mapping-1;return;}
2 int Find(int x) {return lower_bound(Mapping+1,Mapping+m+1,x)-Mapping;}
3
4 struct node {
5     int l,r,LeftSon,RightSon;
6     long long Min,MinSum;
7 }Node[MAXN<<2];
8
9 void Update(int Now) {Node[Now].Min=min(Node[LSon].Min,Node[RSon].Min);
    Node[Now].MinSum=min(Node[LSon].MinSum,Node[RSon].MinSum);return;}
```

正解 100Pts: 建树

```
1 int Build(int l,int r) {  
2     int Now=++cnt;  
3     Node[Now]={l,r,0,0,INF,INF};  
4     if (l==r) return Now;  
5     int Mid=(l+r)>>1;  
6     LSon=Build(l,Mid);RSon=Build(Mid+1,r);  
7     return Now;  
8 }
```

正解 100Pts: 单点插入

```
1
2 void Modify(int Now,int Pos,long long Val) {
3     if (Node[Now].l==Node[Now].r) {
4         Node[Now].Min=min(Node[Now].Min,Val);
5         Node[Now].MinSum=min(Node[Now].MinSum,Val+Mapping[Node[Now].l]);
6         return;
7     }
8     int Mid=(Node[Now].l+Node[Now].r)>>1;
9     if (Pos<=Mid) Modify(LSon,Pos,Val);
10    else Modify(RSon,Pos,Val);
11    Update(Now);return;
12 }
```

正解 100Pts: 查询转移

```
1 long long Query(int Now,int l,int r,int Type) {
2     if (l>r||Node[Now].l>r||Node[Now].r<l) return INF;
3     if (Node[Now].l>=l&&Node[Now].r<=r) return Type?Node[Now].MinSum:
        Node[Now].Min;
4     return min(Query(LSon,l,r,Type),Query(RSon,l,r,Type));
5 }
6
7 long long Calc(int Val) {
8     int Pos=Find(Val);
9     return Tag+min(Query(1,1,Pos,0)+Val,Query(1,Pos+1,m,1));
10 }
```

正解 100Pts: 逐批处理

```
1    Discrete();
2    Build(1,m);
3    int Pos=Find(Line[1]);
4    Modify(1,Pos,0);
5    for (int i=3;i<=n;i+=2) {
6        long long Left=Calc(Line[i]);
7        long long Right=Calc(Line[i-1]);
8        Tag+=max(Line[i],Line[i-1]);
9        Pos=Find(Line[i-1]);
10       Modify(1,Pos,Left-Tag);
11       Pos=Find(Line[i]);
12       Modify(1,Pos,Right-Tag);
13   }
14   printf("%lld\n",Tag+Node[1].MinSum);
```

Contents

T1 我流奥义 (smoke)

T2 星语楠尘 (stardust)

T3 日月风随 (solwind)

T4 推翻暴政 (revolt)

吐槽时间